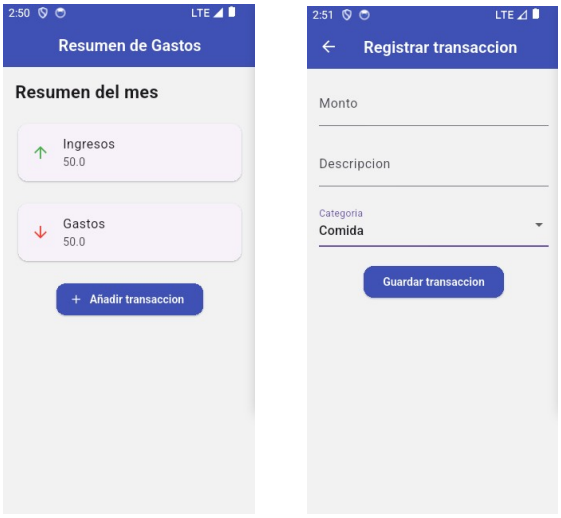
1. Creación de un nuevo proyecto en Flutter

Crearemos una app funcional para gestionar gastos. Incluye gráficas y SQLite. La figura de abajo son algunos screenshots.



2. Prototipo versión 1

En Drive se encuentra la aplicación funcionando. Sin embargo, aún faltan algunas funcionalidades. Esta versión incluye el screen del home con 2 tarjetas y un screen del form que permite cargar ingresos, gastos y categorizarlos.

3. Prototipo versión 2

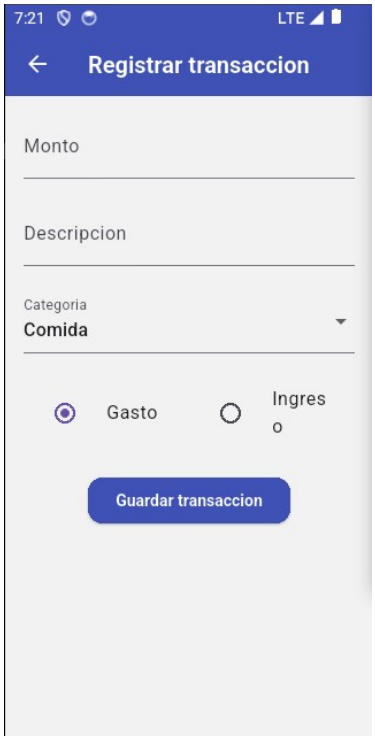
Esta nueva versión incluya permite visualizar gastos, cargar gastos e ingresos y ver historial. Ver Drive. Para la próxima versión se incluye SQLite.

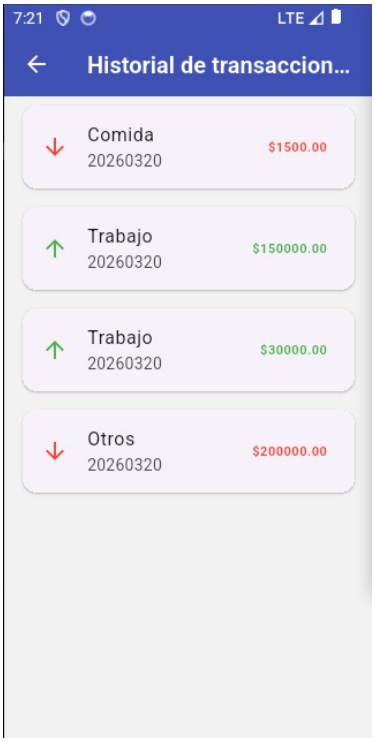
1. main.dart: punto de entrada y configuración global de la app

Qué hace

- Inicializa la app (runApp)
- Configura temas







2. expense_data.dart: Modelo simple para visualización

Qué hace

- Representa datos agregados para gráficos

Partes relevantes:

```
class ExpenseData{
  final String category;
  final double amount;
}
```

3. transaction.dart: Modelo de dominio

Qué hace

- Representa una transacción financiera

Partes relevantes:

- Enum tipado
  `enum TransactionType { income, expense }`
- Entidad
  `class Transaction`

4. transaction_provider.dart: Gestión de estado

Qué hace

- Gestiona el estado global de transacciones

Partes relevantes:

- Lista privada
  `final List<Transaction> _transactions = [];`

- Getter público
  `List<Transaction> get transactions => _transactions;`
- Mutaciones con notificación
  `notifyListeners();`
- Operaciones
  `addTransaction()`

  `deleteTransaction()`

5. summary_screen.dart: Pantalla Ppal

Qué hace

- Pantalla de resumen (ingresos, gastos, gráfico)

Partes relevantes:

- Acceso a estado
  `final transactionProvider = Provider.of<TransactionProvider>(context);`
- Cálculo
  `fold(…)`
- Composición UI: Cards, ExpenseChart y navegación a otras pantallas

6. transaction_form_screen.dart: Formulario de entrada

Qué hace

- Permite crear nuevas transacciones

Partes relevantes:

- Form + validación
  `GlobalKey<FormState>()`
- Controllers
  `TextEditingController`
- Estado local
  _selectedCategory
  _selectedType
- UUID
  Uuid().v4()

7. transaction_history_screen.dart: histórico

Qué hace

- Muestras todas las transacciones registradas

Partes relevantes:

- ListView.builder
- Feedback + acción
  `onLongPress → deleteTransaction()`

8. expense_chart.dart: visualización de datos

Qué hace

- Renderiza gráfico de barras por categoría

Partes relevantes:

- Filtrado
  `where(type == expense)`
- Agregación manual
  `Map<String,double> categoryTotals`
- fl_chart: BarChart, Tooltips, AxisTitles

4. Prototipo versión 3

Esta última versión incluye filtros, se pueden agregar y actualizar tareas. Además, los datos se almacenan en SQLite. Ver en Drive.

1. main.dart

Qué hace

- Inicializa la app, inyecta dependencias (Provider) y define navegación global

Mejora respecto a versiones anteriores:

- `Uso de AppRoutes`
- `Navegación desacoplada`
- Escalable

2. app_routes.dart: Gestión centralizada de navegación

Qué hace

- Define rutas y evita navegación harcodeada

Partes relevantes:

- Constantes
  ```
  static const home = '/';
  ```
- `Map<String, WidgetBuilder>`

La versión anterior de la app:

- Tenía `MaterialPageRoute` repetido en cada pantalla

3. expense_data.dart

Qué hace

- Representa datos agregados por categoría

4. transaction.dart

Qué hace

- Representa una transacción financiera y permite persistencia

Partes relevantes:

- `toMap()` → clave para SQLite

Mejora respecto a versiones anteriores:

- Antes, sólo modelo en memoria.
- Ahora, serialización explícita → persistencia
- Compatible con DB

4. transaction_provider.dart: Gestor de estado + capa intermedia con base de datos

Qué hace

- Mantiene estado en memoria
- Sincroniza con SQLite

Partes relevantes:

- `_transactions`
- `_dbHelper`
- `Métodos async:`
  - `loadTransactions`
  - `addTransaction`
  - `updateTransaction`

Mejora respecto a versiones anteriores:

- Antes, sólo en memoria. Se perdían los datos al cerrar app.
- Ahora, persistencia con SQLite
- Sincronización UI → DB

Algunos detalles más:

- Por qué `Future<void>` y no `void`
  - Motivo técnico: interactúa con IO (DB)

- La versión anterior `void addTransaction() async`
  - No se puede controlar errores
  - Riesgo de inconsistencia

- `await provider.addTransaction()`
  - Permite control de flujo
  - Permite mostrar Snackbar luego de persistir

5. summary_screen.dart

Qué hace

- Mostrar resumen financiero y acceso a funcionalidades

Partes relevantes:

- `initState` → carga inicial
- `context.watch()` → reactividad
- Cálculo:
  - ingresos
  - egresos

Mejora respecto a versiones anteriores:

- Antes, no había carga persistente
- Ahora, se inicializa desde DB
- Estado consistente al abrir la app

6. transaction_form_screen.dart

Qué hace

- Formulario de creación/edición de transacciones

Partes relevantes:

- `_formKey`
- Controllers
- `async onPressed`
- Uso de `context.read()`

Mejora respecto a versiones anteriores:

- Antes, sin persistencia. Lógica duplicada.
- Ahora, uso correcto de async
- UI espera persistencia

7. transaction_history_screen.dart

Qué hace

- Listado histórico de transacciones

Mejora respecto a versiones anteriores:

- Antes, sólo visualización
- Ahora, CRUD completo
- Ahora integrado con DB

8. database_helper.dart: Capa de acceso a datos

Qué hace

- Encapsula toda la lógica de persistencia

Partes relevantes:

- Singleton: `static final DatabaseHelper _instance`
  Evita múltiples conexiones

- Lazy initilization: `Future<Database> get database async`
  DB se crea sólo cuando se necesita

Algunos detalles más:

- Un `Future<T>` es un valor que estará disponible en el futuro (operación async)
- `final db = await database;`
  - Si la DB no existe, se crea
  - Si existe, se reutiliza
  - Todo sin bloquear la UI
- Operaciones async:
  - Insert: `await db.insert()`
  - Query: `final maps = await db.query()`
  - Update: `await db.update()`
  - Delete: `await db.delete()`

Importante:
Las operaciones en TransactionProvider y DatabaseHelper son async porque involucran IO (entrada/salida):

- Acceso a base de datos SQLite
- Lectura/escritura en disco

Y si fueran sync?

```
_transactions = _dbHelper.getTransactions(); // hipotético sync
```

Qué pasaría?

- Se bloquearía el thread principal
- La UI de congela
- Mala UX

Casos concretos para analizar

1. DatabaseHelper

```
Future<List<Transaction>> getTransactions() async {
  final db = await database;
  final maps = await db.query('transactions');
}
```

Tenemos dos operaciones async:

- `await database` → abrir/conectar DB
- `await db.query()` → ejecutar query

Ambas:

- Acceden a disco
- Son operaciones lentas
- Deben no ser bloqueantes

2. TransactionProvider

```
Future<void> addTransaction(Transaction transaction) async {
  _transactions.add(transaction);              // rápido (memoria)
  await _dbHelper.insertTransaction(transaction); // lento (I/O)
  notifyListeners();
}
```

Observación:

- _transaction.add() → sincrónico
- `_dbHelper.insertTransaction()` → **async**

Por qué **await** es importante y no opcional?
Haciendo esto:

```
_dbHelper.insertTransaction(transaction); // sin await
notifyListeners();
```

Problemas:

- La UI se actualiza antes que la DB termine
- Puede haber inconsistencia, errores silenciosos, etc.

Con **await**:

- Se garantiza consistencia
- Orden lógico correcto
- Mejor manero de errores

Regla de oro:

- TODO lo que sea I/O → async
- TODO lo que toque disco/red → async

Se usan operaciones async porque:

- Acceden a recursos lentos (DB/disco)
- Flutter necesita mantener la UI fluida
- Dart usa Future para modelar latencia
- await garantiza orden y consistencia

9. expense_chart.dart

Qué hace

- Visualización de gastos

En este momento, la app tiene esta separación de capas

UI → Provider → DatabaseHelper → DB